🎯 Guía de Migración a TanStack Query

📊 Estado Actual

DevTools vacío porque solo 5 módulos usan TanStack Query:

- ✅ `useOrders` - Órdenes
- ✅ `useWishlistQuery` - Lista de deseos
- ✅ `useProducts` / `useCategories` - Productos/Categorías
- ✅ `useAdminDashboard` - Dashboard admin

El resto usa `useState` + `useEffect` + `fetch` ❌

🔴 Casos Críticos para Migrar (ALTA PRIORIDAD)

1. **useCart** (Carrito)

Problema: 200+ líneas de `useState`, `useEffect`, `usePersistentState`
Archivo: `frontend/src/modules/cart/hooks/useCart.js`

Beneficios de migrar:

- ✅ Sincronización automática entre componentes
- ✅ Optimistic updates (UI instantánea)
- ✅ Rollback automático en errores
- ✅ Menos código (de 200 a ~100 líneas)

Migración: Ya creado `useCartQuery.js` - ver ejemplo

---

2. **useRegionesYComunas** (Regiones/Comunas)

Problema: Fetch manual sin caché, re-fetch innecesario
Archivo: `frontend/src/hooks/useRegionesYComunas.js`

Beneficios de migrar:

- ✅ Caché de 1 hora (regiones NO cambian)
- ✅ Carga condicional de comunas (solo si hay región)
- ✅ Menos re-renders

Migración: Ya creado `useRegionesYComunas.query.js` - ver ejemplo

---

3. **useUser** (Perfil de Usuario)

Problema: `useState` + `axios` manual, no sincroniza con AuthContext
Archivo: `frontend/src/modules/profile/hooks/useUser.js`

Beneficios de migrar:

- ✅ Invalidación automática al actualizar perfil
- ✅ Sincronización con `AuthContext`
- ✅ Caché de 5 minutos

Migración: Ya creado `useUserProfile.js` - ver ejemplo

📝 Pasos para Migrar un Hook

Ejemplo: Migrar `useRegionesYComunas.js`

❌ Antes (useState + useEffect):

```
const [regiones, setRegiones] = useState([]);
const [loading, setLoading] = useState(false);

useEffect(() => {
  const fetchRegiones = async () => {
    setLoading(true);
    const res = await fetch('/api/regiones');
    const data = await res.json();
    setRegiones(data.data);
    setLoading(false);
  };
  fetchRegiones();
}, []);
```

✅ Después (TanStack Query):

```
const query = useQuery({
  queryKey: ['regiones'],
  queryFn: async () => {
    const res = await fetch('/api/regiones');
    const data = await res.json();
    return data.data;
  },
  staleTime: 60 * 60 * 1000, // 1 hora
});

const regiones = query.data ?? [];
const loading = query.isLoading;
```

Ventajas:

- 📦 Menos código (de ~30 líneas a ~10)
- 🚀 Caché automático (sin re-fetch innecesarios)
- 🔄 Retry automático en errores
- 📊 Visible en DevTools

🛠️ Checklist de Migración

Hooks de Datos (useQuery)

- [ ] `useCart` → `useCartQuery` ⭐ **ALTA PRIORIDAD**
- [ ] `useRegionesYComunas` → usar `.query.js` ⭐
- [ ] `useUser` → `useUserProfile` ⭐
- [ ] `useAdminCustomers` (si existe)
- [ ] `useAdminStats` (dashboard stats)
- [ ] `usePaymentMethods`
- [ ] `useShippingMethods`

Hooks de Acciones (useMutation)

- [ ] `addToCart` → `useAddToCart` ⭐
- [ ] `updateProfile` → `useUpdateProfile` ⭐
- [ ] `createOrder` → ya existe en `useOrders`
- [ ] `updateOrderStatus` (admin)
- [ ] `createProduct` (admin)
- [ ] `updateProduct` (admin)

🎓 Patrones Comunes

1. Caché de Datos Estáticos (1 hora+)

```
// Regiones, métodos de pago, categorías
staleTime: 60 * 60 * 1000, // 1 hora
cacheTime: 24 * 60 * 60 * 1000, // 24 horas
```

2. Caché de Datos Dinámicos (2-5 min)

```
// Carrito, órdenes, perfil
staleTime: 2 * 60 * 1000, // 2 minutos
```

```
cacheTime: 10 * 60 * 1000, // 10 minutos
```

3. Queries Condicionales

```
// Solo fetch si hay usuario autenticado
enabled: Boolean(userId),
```

4. Optimistic Updates (Mutations)

```
onMutate: async (newData) => {
  // 1. Cancelar queries en vuelo
  await queryClient.cancelQueries({ queryKey: ['cart'] });

  // 2. Guardar snapshot para rollback
  const previous = queryClient.getQueryData(['cart']);

  // 3. Actualizar caché inmediatamente (optimistic)
  queryClient.setQueryData(['cart'], (old) => [...old, newData]);

  return { previous }; // Para rollback
},
onError: (err, vars, context) => {
  // Rollback si falla
  queryClient.setQueryData(['cart'], context.previous);
},
onSuccess: () => {
  // Invalidar para refetch real
  queryClient.invalidateQueries({ queryKey: ['cart'] });
}
```

🧪 Cómo Verificar la Migración

1. **Abre el DevTools** (icono flotante)
2. **Navega en la app** (home → productos → carrito → perfil)
3. **Deberías ver queries activas:**
   - `['regiones']`
   - `['comunas', 'RM']`
   - `['cart']`
   - `['user', 'profile', 123]`
   - `['products', { limit: 20, page: 1 }]`

4. **Haz clic en una query** para ver:
    - ✅ Estado: `success` / `loading` / `error`
    - 🕐 Last Updated: hace X segundos
    - 📊 Data: JSON completo
    - 🔄 Observers: cuántos componentes la usan

📈 Beneficios Medibles

Antes (sin Query)

- ❌ 3-5 fetches duplicados al navegar
- ❌ Loading spinner cada vez que cambias de página
- ❌ Estado no sincronizado entre componentes
- ❌ No hay retry en errores de red

Después (con Query)

- ✅ 1 solo fetch inicial (resto desde caché)
- ✅ Transiciones instantáneas (UI usa caché)
- ✅ Estado global sincronizado
- ✅ Retry automático (2 intentos)
- ✅ Invalidación inteligente al mutar

🚀 Próximos Pasos

1. **FASE 1 - Migrar hooks críticos:**
    - `useCart` → `useCartQuery` (mayor impacto)
    - `useRegionesYComunas` → usar `.query.js`
    - `useUser` → `useUserProfile`
2. **FASE 2 - Actualizar componentes:**
    - Buscar imports de hooks viejos
    - Reemplazar por hooks nuevos
    - Remover `useState`/`useEffect` innecesarios
3. **FASE 3 - Testing:**
    - Verificar DevTools muestra queries
    - Probar optimistic updates en carrito
    - Verificar caché funciona (sin re-fetch innecesarios)
4. **FASE 4 - Cleanup:**

    - Eliminar hooks viejos
    - Actualizar documentación
    - Commit con mensaje descriptivo

🔗 Recursos

- **Docs oficiales:** https://tanstack.com/query/latest
- **DevTools:** Icono flotante en app (dev mode)
- **Ejemplos en proyecto:**
    - `useOrders.js` - Queries + mutations completas
    - `useWishlistQuery.js` - Queries con invalidación
    - `useProducts.js` - Queries con filtros/paginación

⚠️ Notas Importantes

1. **No migrar todo de golpe** - Hazlo por módulos
2. **Probar en Dev primero** - Verificar con DevTools
3. **Mantener backward compatibility** - Ambos hooks pueden coexistir temporalmente
4. **Documentar cambios** - Actualizar imports en otros archivos

---

¿Preguntas? Revisa los archivos ejemplo creados:

- `useRegionesYComunas.query.js`
- `useUserProfile.js`
- `useCartQuery.js`